Machine-Level Programming: Memory Addressing Modes

Arquitectura de Computadores
Departamento de Engenharia Informática
Instituto Superior de Engenharia do Porto

Luís Nogueira (lmn@isep.ipp.pt)

Today: Memory Addressing Modes

- **Simple addressing mode**
- **Complete addressing mode**
- **Address computation with *leal***

Accessing memory

- **Variables are stored in memory**
  - Global and static local variables in *data* or *bss* sections
  - Dynamically allocated variables in the heap
  - Function parameters and local variables on the stack
- **Need to be able to load from and store to memory**
  - To manipulate the data directly in memory or copy the data between main memory and registers
- **IA-32 has many different addressing modes**
  - Corresponding to common programming constructs

Direct addressing

- **Load or store from a particular memory location**
  - **Memory address** is embedded in the instruction
  - Instruction reads from or writes to that address
- **Example: `movl 2000, %ecx`**
  - Read four bytes starting at address 2000
  - Load the value into the ECX register
  - Notice the missing $ in front of 2000
- **Useful when the address is known in advance**

Direct addressing

- **Most commonly used with symbolic links to specify the address instead of numbers for human readability**
  - E.g., global variables in the *data* or *bss* sections

- **Example: `addl num1, %eax`**
  - Adds the contents pointed by memory address `num1` to EAX
  - The Assembler converts the symbolic name `num1` to the corresponding address

Indirect addressing

- **Load or store from a previously-computed address**
  - **Register with the address** is embedded in the instruction
- **Example: `movl $num1, %eax`**
  - EAX stores the 32-bit base address (e.g. 2000) of `num1`
- **A subsequent instruction reads from or writes to the address stored in register**
- **Example: `movl (%eax), %ecx`**
  - Read four bytes starting at address pointed by EAX (e.g. 2000)
  - Load the value into the ECX register

Simple memory addressing modes

- **Normal (R) Mem[Reg[R]]**
  - Register R specifies memory address

```
movl (%ecx),%eax
```

- **Displacement D(R) Mem[Reg[R]+D]**
  - Register R specifies start of memory region
  - Constant displacement D specifies offset

```
movl 8(%ebp),%edx
```

Using simple addressing modes

C code

```
void swap(int *xp, int *yp)
{
  int t0 = *xp;
  int t1 = *yp;
  *xp = t1;
  *yp = t0;
}
```

Assembly code

```
swap:
  pushl %ebp
  movl  %esp,%ebp
  pushl %ebx

  movl  8(%ebp),%edx
  movl  12(%ebp),%ecx
  movl  (%edx),%ebx
  movl  (%ecx),%eax
  movl  %eax,(%edx)
  movl  %ebx,(%ecx)

  popl  %ebx
  popl  %ebp
  ret
```

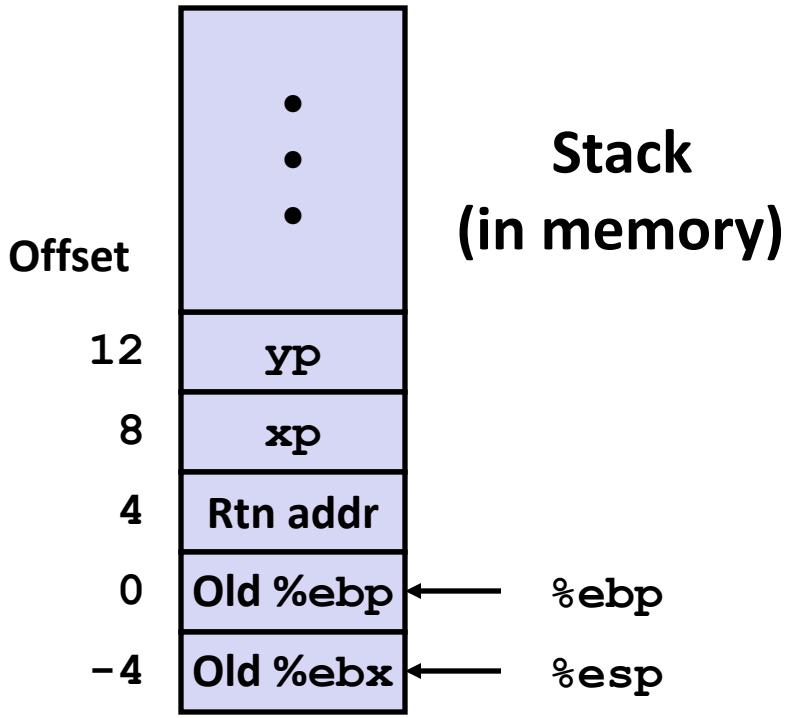
Set Up

Body

Finish

Understanding *swap()*

```
void swap(int *xp, int *yp)
{
  int t0 = *xp;
  int t1 = *yp;
  *xp = t1;
  *yp = t0;
}
```



Register	Value
%edx	xp
%ecx	yp
%ebx	t0
%eax	t1

```
movl   8(%ebp),%edx     # edx = xp
movl   12(%ebp),%ecx    # ecx = yp
movl   (%edx),%ebx      # ebx = *xp (t0)
movl   (%ecx),%eax      # eax = *yp (t1)
movl   %eax,(%edx)      # *xp = t1
movl   %ebx,(%ecx)      # *yp = t0
```

Understanding *swap()*

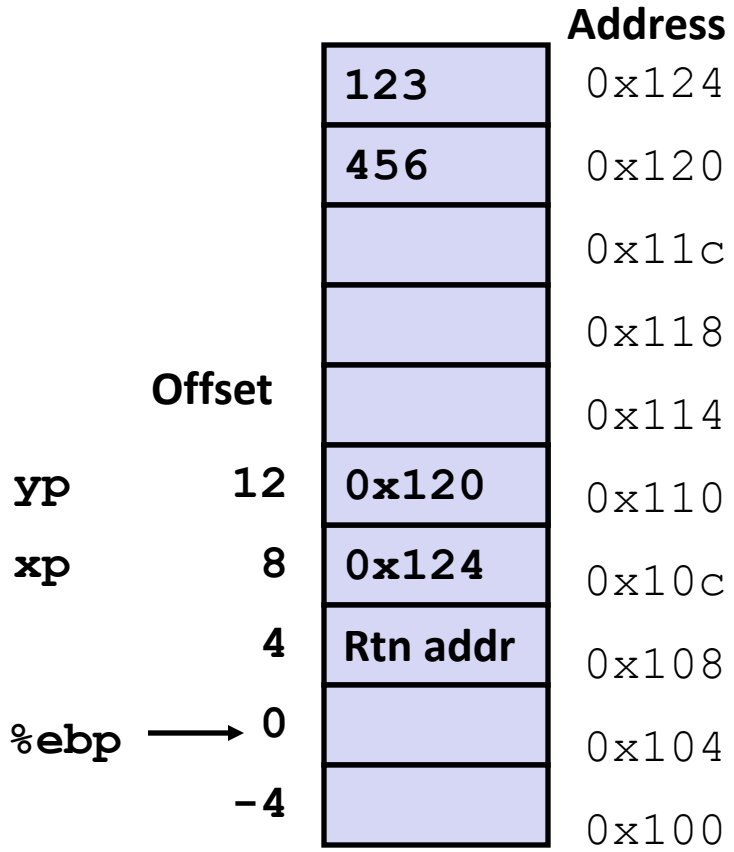
%eax	
%edx	
%ecx	
%ebx	
%esi	
%edi	
%esp	0x100
%ebp	0x104

	Offset		Address
		123	0x124
		456	0x120
			0x11c
			0x118
			0x114
yp	12	0x120	0x110
xp	8	0x124	0x10c
	4	Rtn addr	0x108
%ebp →	0		0x104
	-4		0x100

```
movl   8(%ebp),%edx      # edx = xp
movl   12(%ebp),%ecx     # ecx = yp
movl   (%edx),%ebx       # ebx = *xp (t0)
movl   (%ecx),%eax       # eax = *yp (t1)
movl   %eax,(%edx)       # *xp = t1
movl   %ebx,(%ecx)       # *yp = t0
```

Understanding *swap()*

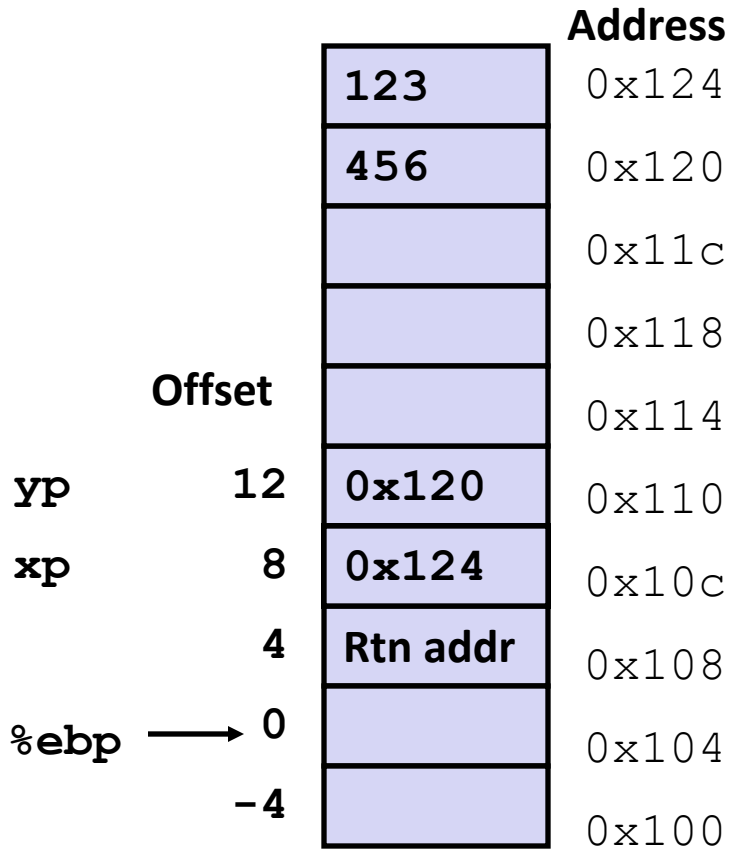
%eax	
%edx	0x124
%ecx	
%ebx	
%esi	
%edi	
%esp	0x100
%ebp	0x104




```
movl   8(%ebp),%edx      # edx = xp
movl   12(%ebp),%ecx     # ecx = yp
movl   (%edx),%ebx       # ebx = *xp (t0)
movl   (%ecx),%eax       # eax = *yp (t1)
movl   %eax,(%edx)       # *xp = t1
movl   %ebx,(%ecx)       # *yp = t0
```

Understanding *swap()*

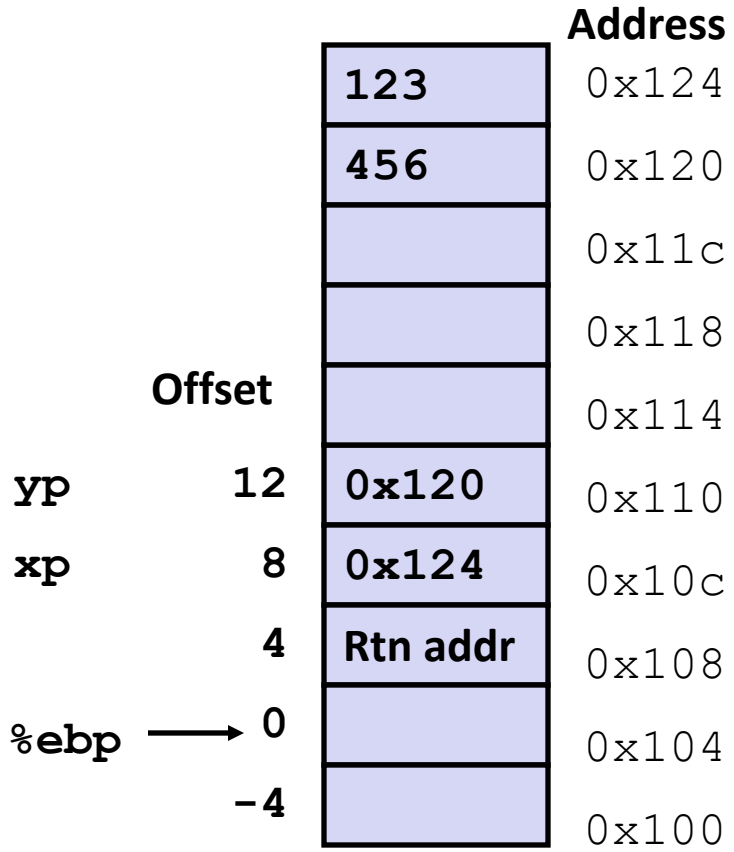
%eax	
%edx	0x124
%ecx	0x120
%ebx	
%esi	
%edi	
%esp	0x100
%ebp	0x104




```
movl   8(%ebp),%edx      # edx = xp
movl   12(%ebp),%ecx     # ecx = yp
movl   (%edx),%ebx       # ebx = *xp (t0)
movl   (%ecx),%eax       # eax = *yp (t1)
movl   %eax,(%edx)       # *xp = t1
movl   %ebx,(%ecx)       # *yp = t0
```

Understanding *swap()*

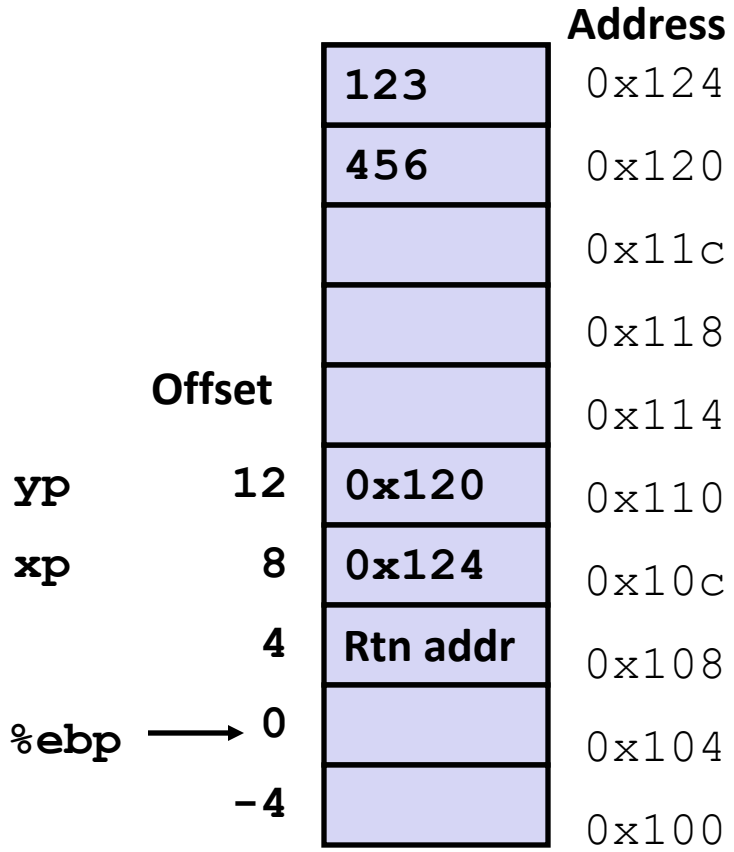
%eax	
%edx	0x124
%ecx	0x120
%ebx	123
%esi	
%edi	
%esp	
%ebp	0x104




```
movl   8(%ebp),%edx      # edx = xp
movl   12(%ebp),%ecx     # ecx = yp
movl   (%edx),%ebx       # ebx = *xp (t0)
movl   (%ecx),%eax       # eax = *yp (t1)
movl   %eax,(%edx)       # *xp = t1
movl   %ebx,(%ecx)       # *yp = t0
```

Understanding *swap()*

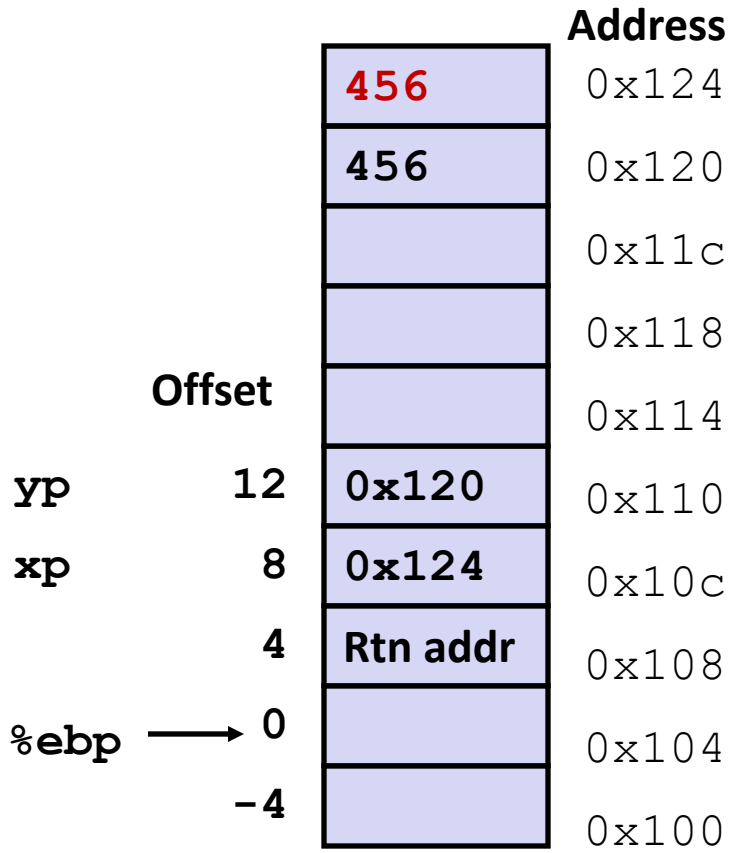
%eax	456
%edx	0x124
%ecx	0x120
%ebx	123
%esi	
%edi	
%esp	
%ebp	0x104




```
movl   8(%ebp),%edx      # edx = xp
movl   12(%ebp),%ecx     # ecx = yp
movl   (%edx),%ebx       # ebx = *xp (t0)
movl   (%ecx),%eax       # eax = *yp (t1)
movl   %eax,(%edx)       # *xp = t1
movl   %ebx,(%ecx)       # *yp = t0
```

Understanding *swap()*

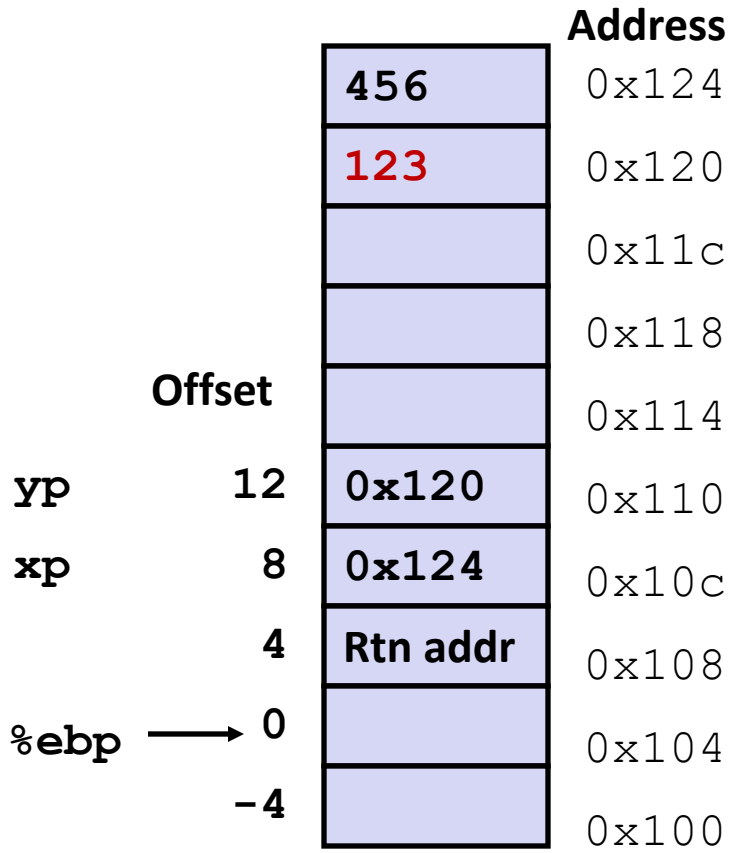
%eax	456
%edx	0x124
%ecx	0x120
%ebx	123
%esi	
%edi	
%esp	
%ebp	0x104




```
movl   8(%ebp),%edx      # edx = xp
movl   12(%ebp),%ecx     # ecx = yp
movl   (%edx),%ebx       # ebx = *xp (t0)
movl   (%ecx),%eax       # eax = *yp (t1)
movl   %eax,(%edx)       # *xp = t1
movl   %ebx,(%ecx)       # *yp = t0
```

Understanding *swap()*

%eax	456
%edx	0x124
%ecx	0x120
%ebx	123
%esi	
%edi	
%esp	
%ebp	0x104




```
movl   8(%ebp),%edx      # edx = xp
movl   12(%ebp),%ecx     # ecx = yp
movl   (%edx),%ebx       # ebx = *xp (t0)
movl   (%ecx),%eax       # eax = *yp (t1)
movl   %eax,(%edx)       # *xp = t1
movl   %ebx,(%ecx)       # *yp = t0
```

Today: Memory Addressing Modes

- Simple addressing mode
- **Complete addressing mode**
- Address computation with *leal*

Complete memory addressing modes

D(Rb,Ri,S) → Mem[Reg[Rb] + D + S*Reg[Ri]]

- **D: Constant "displacement" in bytes**
- **Rb: Base register: Any of the 8 integer registers**
- **Ri: Index register: Any, except for `%esp`**
  - Unlikely you'd use `%ebp`, either
- **S: Scale: 1, 2, 4, or 8 (why these numbers?)**

Complete memory addressing modes

- **Special cases**

(Rb,Ri)	**→**	**Mem[Reg[Rb] + Reg[Ri]]**
D(Rb,Ri)	**→**	**Mem[Reg[Rb] + Reg[Ri] + D]**
(Rb,Ri,S)	**→**	**Mem[Reg[Rb] + S*Reg[Ri]]**

- **Notes:**
  - If any of the values are zero, they can be omitted (but the commas are still required as placeholders)
  - If we omit the base address, the offset can be an absolute address

Address computation examples (1/3)

`%edx`	`0xF000`
`%ecx`	`0x0100`

Expression	Address Computation	Address
`0x8(%edx)`	`0xF000 + 0x8`	`0xF008`
`(%edx,%ecx)`	`0xF000 + 0x100`	`0xF100`
`(%edx,%ecx,4)`	`0xF000 + 4*0x100`	`0xF400`
`0x80(,%edx,2)`	`2*0xF000 + 0x80`	`0x1E080`

Address computation examples (2/3)

C code

```
int *ptr, vec[] = {1,2,3};

void f()
{
  ptr = &vec[2];
  *ptr = 10;
  …
}
```

Assembly code

```
f:
  movl   $vec,%esi                # esi = &vec[0]
  movl   $2,%ecx
  movl   $10,(%esi,%ecx,4)        # vec[2] = 10
  …
  ret
```

Address computation examples (3/3)

C code

```
int *ptr, vec[] = {1,2,3};

void f()
{
  ptr = &vec[2];
  *ptr = 10;
  …
}
```

Assembly code

```
f:
  movl   $2,%ecx
  movl   $10,vec(,%ecx,4)          # vec[2] = 10
  …
  ret
```

Today: Memory Addressing Modes

- Simple addressing mode
- Complete addressing mode
- **Address computation with *leal***

Address computation instruction: *leal*

`leal` Src,Dest → Dest = &Src

- **Load Effective Address**
  - Src is an address mode expression
  - Set Dest (must be a register) to address denoted by expression
- **Uses**
  - Computing addresses **without a memory reference**
    - E.g., translation of `p = &x[i];`
  - Computing arithmetic expressions of the form x + k*y
    - k = 1, 2, 4, or 8

Address computation instruction: *leal*

C code

```
int *ptr, vec[] = {1,2,3};

void f()
{
  ptr = &vec[2];
  *ptr = 10;
  …
}
```

Assembly code

```
f:
  leal   vec,%esi                  # esi = &vec[0]
  movl   $2,%ecx
  leal   (%esi,%ecx,4),%esi        # esi = &vec[2]
  movl   $10,(%esi)
  …
  ret
```

Computing arithmetic expressions with *leal*

C code

```
long mul_12(long x)
{
  return x*12;
}
```

Compiled arithmetic operations

```
  leal (%eax,%eax,2),%eax
  shll $2,%eax
```

Explanation

```
  t <- x + x*2
  return t << 2;
```

- **The compiler takes advantage of the addressing circuitry and uses it to do arithmetic**
  - As opposed to generating multiple arithmetic instructions to do arithmetic

Another example

C code

```
int arith(int x, int y, int z)
{
  int t1 = x + y;
  int t2 = z + t1;
  int t3 = x + 4;
  int t4 = y * 48;
  int t5 = t3 + t4;
  int rval = t2 * t5;
  return rval;
}
```

Assembly code

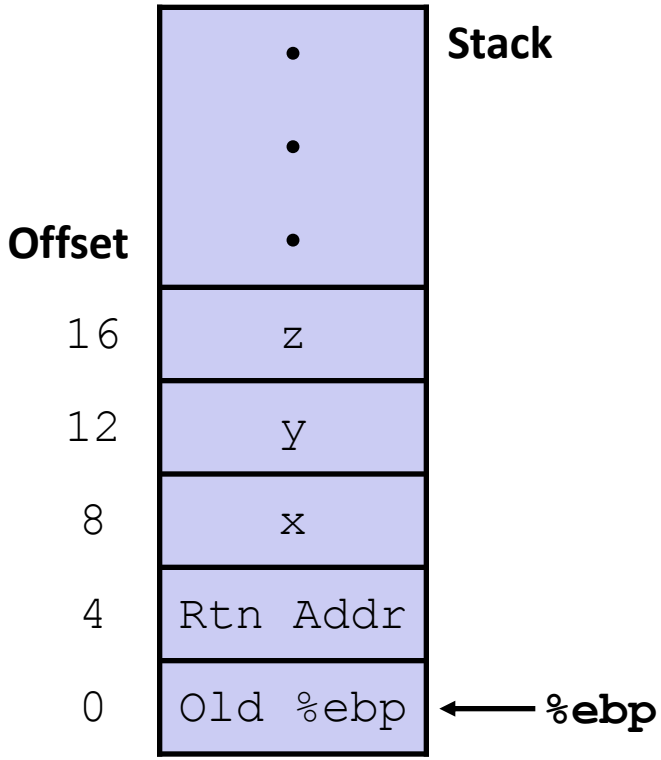
```
arith:
  pushl   %ebp
  movl    %esp,%ebp

  movl    8(%ebp),%ecx
  movl    12(%ebp),%edx
  leal    (%edx,%edx,2),%eax
  sall    $4,%eax
  leal    4(%ecx,%eax),%eax
  addl    %ecx,%edx
  addl    16(%ebp),%edx
  imull   %edx,%eax

  movl    %ebp,%esp
  popl    %ebp
  ret
```

Understanding *arith()*

```
int arith(int x, int y, int z)
{
  int t1 = x+y;
  int t2 = z+t1;
  int t3 = x+4;
  int t4 = y * 48;
  int t5 = t3 + t4;
  int rval = t2 * t5;
  return rval;
}
```




```
movl   8(%ebp),%ecx          # %ecx = x
movl   12(%ebp),%edx         # %edx = y
leal   (%edx,%edx,2),%eax    # %eax = y*3
sall   $4,%eax               # %eax *= 16          (t4)
leal   4(%ecx,%eax),%eax     # %eax = t4+x+4       (t5)
addl   %ecx,%edx             # %edx = x+y          (t1)
addl   16(%ebp),%edx         # %edx += z           (t2)
imull  %edx,%eax             # %eax = t2 * t5      (rval)
```

Understanding *arith()*

```
int arith(int x, int y, int z)
{
  int t1 = x+y;
  int t2 = z+t1;
  int t3 = x+4;
  int t4 = y * 48;
  int t5 = t3 + t4;
  int rval = t2 * t5;
  return rval;
}
```

- Instructions in different order from C code
- Some expressions require multiple instructions
- Some instructions cover multiple expressions
- Get exact same code when compile **`(x+y+z)*(x+4+48*y)`**

```
movl   8(%ebp),%ecx          # %ecx = x
movl   12(%ebp),%edx         # %edx = y
leal   (%edx,%edx,2),%eax    # %eax = y*3
sall   $4,%eax               # %eax *= 16          (t4)
leal   4(%ecx,%eax),%eax     # %eax = t4+x+4       (t5)
addl   %ecx,%edx             # %edx = x+y          (t1)
addl   16(%ebp),%edx         # %edx += z           (t2)
imull  %edx,%eax             # %eax = t2 * t5      (rval)
```

Memory Addressing Modes: Summary

- **The most general addressing mode has syntax D(Rb,Ri,S)**
  - The other forms are simply special cases of this general form where some of the components are omitted

- ***leal* looks like a *mov instruction*, but does not access memory**
  - Instead, it takes advantage of the addressing circuitry and uses it to do arithmetic
  - Compilers often find clever uses of *leal* that have nothing to do with effective address computations